

1 External Security Notes

1.1 Deployment Recommendations

We recommend a simple but safe deployment setup:

- Keep the five storage servers on a private network and do not expose them directly to the public internet.
- Use HTTPS through a reverse proxy if clients are not running on localhost.
- Set a strong shared API token and rotate it if it is leaked.
- Run each server process with limited filesystem permissions so it can only access its own data folder.
- Turn on request logging so failed verification events are easy to trace during debugging and demos.

1.2 Known Limitations

Veri-Store is primarily an *integrity* prototype, not a complete secure storage system. Its verification logic is effective at detecting fragment corruption and many forms of byzantine tampering, but several important security properties remain outside the current design.

- **No confidentiality guarantees.** Fragments are stored and transmitted as plaintext unless the integrating application adds encryption or deploys the system behind TLS. Erasure coding alone does not make the data confidential.
- **The client does not retrieve against a separately trusted FPCC.** During GET, the client currently adopts the first successfully returned `fpcc_json` as the reference value for fragment verification. This means the client proves that fragments are internally consistent with that `fpcc`, but not that the `fpcc` itself came from a trusted source. If at least m malicious servers return the same forged `fpcc` and mutually consistent fragments, the client may reconstruct attacker-chosen data.
- **Block identity is not cryptographically bound into verification.** The current `fpcc` commits to fragment hashes and fingerprints, but not to the application-level `block_id`. As shown by the penetration-testing results (`.\docs\experiments\pentest-attack-matrix.md`), a valid fragment or a full stale response can be replayed under a different block namespace and still pass the low-level fragment checks. This is an accepted limitation of the current prototype and would need to be addressed before claiming stronger object-authenticity guarantees.
- **Authentication is deployment-wide rather than user-specific.** The bearer token protects the service from unauthenticated access, but any party that learns the shared token can read, write, or delete any block whose identifier it knows. There is no per-user authorization, ownership check, or audit identity binding in the current implementation. This is an accepted risk, as the current implementation is meant for a single user, but would need to be mitigated for a multi-user deployment.
- **Security depends on threshold assumptions.** The integrity story assumes that fewer than m servers collude maliciously when the client is relying on server-supplied metadata. The system also assumes the cryptographic primitives behave as expected, in particular the collision resistance of

SHA-256 and the random-oracle-style behavior used in the fingerprint construction. The collision resistance is empirically demonstrated in `.\docs\experiments\collision-probability.md`.

- **Availability and denial-of-service resistance are limited.** Veri-Store can tolerate missing or corrupted fragments up to the reconstruction threshold, but it does not currently provide strong rate limiting (it does have basic rate limiting), storage quotas, admission control, or recovery orchestration suitable for adversarial production environments.

These limitations do not invalidate the prototype’s main goal, which is to demonstrate verified integrity checking for erasure-coded fragments. They do, however, define the boundary of the security claims that this document can make: Veri-Store currently offers useful corruption detection and threshold-based integrity checks, but not full end-to-end authenticity, confidentiality, or production-grade access control.

1.3 Integration Considerations

When Veri-Store is embedded into a larger application, it should be treated as a specialized *integrity-preserving fragment store*, not as a complete secure storage platform. The integrating system must therefore be explicit about which security properties are provided by Veri-Store itself and which are still the responsibility of the surrounding application or deployment environment.

- **What Veri-Store provides.** Veri-Store provides threshold-based fragment storage, deterministic fragment verification against an `fpcc`, and rejection of many corrupted or malformed fragment responses. In the normal honest-client workflow, it helps ensure that reconstructed data has not been silently modified by a single faulty or byzantine storage node.
- **What the integrating system must provide.** The surrounding system must still provide end-user authentication, authorization, object naming policy, confidentiality, key management if encryption is used, transport security, deployment hardening, and operational monitoring. Veri-Store should be viewed as one component in a broader security architecture rather than as the architecture itself.
- **Trusted metadata handling.** If the application needs stronger authenticity guarantees than the current prototype provides, it should retain the original client-generated `fpcc` in a trusted location and use that value during retrieval, rather than trusting the first server-supplied `fpcc`. A production integration may also bind the `fpcc` to an authenticated object manifest, digital signature, or MAC-protected metadata service.
- **Confidentiality layering.** Applications that store sensitive data should encrypt objects before passing them to Veri-Store. In that model, Veri-Store protects the integrity and recoverability of ciphertext fragments, while confidentiality would need to be enforced by an external encryption layer.
- **Identifier semantics.** The current implementation treats `block_id` as an application-level routing key rather than a cryptographically authenticated part of the fragment commitment. Integrating systems should therefore generate block identifiers carefully, avoid exposing predictable identifiers unnecessarily, and avoid assuming that block identity is currently protected to the same degree as fragment contents.
- **Failure handling.** The integrating system should treat retrieval errors, verification failures, and repeated byzantine detections as security-significant events rather than ordinary application errors. These events should be surfaced to logs, monitoring, and incident-response workflows instead of being silently retried without investigation.

In short, Veri-Store fits best as a lower-layer integrity service. It can strengthen a larger system’s resilience to fragment corruption and some classes of byzantine storage behavior, but it relies on the integrating system to supply the rest of the security envelope.